

Reliable Explainable AI Framework for Software Defect Prediction

Submitted in partial fulfilment of the requirements of the degree of

Master of Technology by

Udit Jain

Roll No. 24CSM1R23

Supervisor:

Dr. Manjubala Bisi

Assistant Professor



Department of Computer Science and Engineering

National Institute of Technology Warangal

Telangana, India

April 2026

Acknowledgement

I would like to express my heartily gratitude towards **Dr. Manjubala Bisi** (Supervisor) for her valuable guidance, supervision, suggestions, encouragement and the help throughout the semester, for the completion of my project work. She kept me going when I was down and gave me the courage to keep moving forward.

I also want to thank evaluation committee for their valuable suggestions and conducting smooth presentation of the project.

(Signature)

Udit Jain

Roll No: 24CSM1R23

Department of Computer Science and Engineering, NIT Warangal

Declaration

I hereby declare that the work presented in this dissertation is the outcome of my own research carried out under the supervision of **Dr. Manjubala Bisi**, Assistant Professor, Department of Computer Science and Engineering, National Institute of Technology, Warangal.

I also declare that to the best of my knowledge, this dissertation has not been submitted either in part or in full for the award of any other degree or diploma at this or any other institution. Wherever the work of others has been used, I have duly acknowledged and properly cited the original sources.

I further affirm that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute.

(Signature)

Udit Jain

Roll No: 24CSM1R23

Date :

Approval Sheet

This Dissertation Work entitled “**Reliable Explainable AI Framework for Software Defect Prediction**” by **Udit Jain (24CSM1R23)** is approved for
the degree of Master of Technology (M.Tech).

Examiners

Supervisor

Dr. Manjubala Bisi

Chairman

NIT Warangal

Certificate

This is to certify that the Dissertation work entitled “**Reliable Explainable AI Framework for Software Defect Prediction**” is a bonafide record of work carried out by **Udit Jain (24CSM1R23)**, submitted to Dr. Manjubala Bisi of the Department of Computer Science and Engineering, in partial fulfilment of the requirements for the award of the degree of Master of Technology at National Institute of Technology, Warangal during the academic year 2024–2025.

(Signature)

Dr. Manjubala Bisi

Assistant Professor

Supervisor

Department of Computer Science and Engineering

NIT Warangal

Abstract

Software defect prediction plays a crucial role in improving software quality by identifying defect-prone modules early in the development cycle. While machine learning models such as Random Forests and other ensemble techniques have shown promising results in predicting software defects, their adoption in practice is often hindered by a lack of trust in model explanations. Existing studies primarily focus on accuracy and interpretability but rarely assess the reliability of these explanations.

This dissertation proposes a **Reliable Explainable AI Framework for Software Defect Prediction** that integrates SHAP (SHapley Additive exPlanations) with novel reliability metrics. In addition to standard measures of generalizability, concordance, and stability, the framework introduces three behavior-grounded metrics: (i) *Gradient-of-Loss Reliability (GLR)* to evaluate alignment between SHAP feature importance and true loss sensitivity, (ii) *ϵ -Perturbation Hit@k* to test whether SHAP-identified top features genuinely affect model loss under small perturbations, and (iii) *Expected Calibration Error (ECE)* to assess the reliability of predicted probabilities.

Experiments were conducted on the NASA PROMISE CM1 dataset using Random Forest classifiers. Results demonstrate that while conventional SHAP explanations achieve high generalizability, their reliability varies across folds and conditions. The proposed framework provides a composite reliability score, offering a more comprehensive and trustworthy evaluation of explainability.

This work bridges the gap between interpretability and reliability in explainable AI for software defect prediction. By validating explanations against model behavior and calibration, the framework enhances confidence for developers and stakeholders, moving closer to trustworthy AI in software engineering.

Keywords: Explainable AI, Reliability, SHAP, Software Defect Prediction, Random Forest, GLR, Hit@k, Calibration.

Contents

Acknowledgement	i
Declaration	ii
Approval Sheet	iii
Certificate	iv
Abstract	v
List of Abbreviations	x
1 Introduction	1
1.1 Context and Motivation	1
1.2 Challenges in Current Systems	1
1.3 Reliability as a Core Requirement	2
1.4 Objectives and Scope of the Dissertation	2
1.5 Expected Contributions	3
1.6 Organization of the Report	3
2 Background	4
2.1 Reliability in Explainable AI	4
2.2 Behavior-Grounded Reliability	4
2.3 Machine Learning for Defect Prediction	5
2.4 Explainability as a Tool for Reliability	5
2.5 Summary	6
3 Related Work	7
3.1 Machine Learning for Software Defect Prediction	7
3.2 Explainable AI in Software Engineering	7
3.3 Reliability of Explanations	8
3.4 Positioning in Recent Literature (2023–2025)	9

3.5	Gaps and Open Questions	9
3.6	Need for This Study	9
3.7	Summary	10
4	Proposed Solution	11
4.1	System Overview	11
4.2	Dataset and Preprocessing	13
4.3	Model Training and Standard Evaluation	15
4.4	Explainability using SHAP	16
4.5	Reliability Assessment	17
4.5.1	Classical Reliability Measures	17
4.5.2	Behavior-Grounded Reliability Metrics	17
4.5.3	Composite Reliability Score	18
4.6	Pipeline Code	18
4.7	Visualization Outputs	18
4.8	Summary	20
5	Results and Analysis	21
5.1	Overall Experimental Setup	21
5.2	Same Project Evaluation (Train = Test)	21
5.2.1	Baseline Results (Same Project)	22
5.2.2	Proposed Results (Same Project)	22
5.2.3	WSR Results (Same Project)	22
5.2.4	Conclusion (Same Project)	23
5.3	Cross Project Evaluation (Train $\neq$ Test)	23
5.3.1	WSR Results (Cross Project)	24
5.3.2	Conclusion (Cross Project)	24
5.4	Summary	24
6	Conclusion	25
6.1	Conclusion	25
6.2	Implications of Findings	26
6.3	Limitations	27
6.4	Final Remarks	27
	References	28

List of Figures

- 4.1 Proposed framework pipeline: dataset preparation, preprocessing, Random Forest training, SHAP-based explanations, and reliability evaluation. 12
- 4.2 The top-10 most influential software metrics in the cross-project combination pc4.arff $\rightarrow$ pc1.arff (AdaBoost), based on the *mean importance* computed on the test set. Larger bars indicate higher average contribution magnitude. . . 19

List of Tables

5.1	Baseline results in the Same Project setting (model-wise means with overall column means).	22
5.2	Proposed results in the Same Project setting (model-wise means with overall column means).	22
5.3	Wilcoxon Signed-Rank (WSR) test results for the Same Project setting (Baseline vs Proposed).	22
5.4	Fold-wise evaluation results under the Baseline approach (AdaBoost: Train = pc4, Test = pc1).	23
5.5	Fold-wise evaluation results under the Proposed approach (AdaBoost: Train = pc4.arff, Test = pc1.arff).	23
5.6	Wilcoxon Signed-Rank (WSR) test results comparing Baseline vs Proposed reliability scores on cross-project dataset pairs using AdaBoost.	24

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
SDP	Software Defect Prediction
XAI	Explainable Artificial Intelligence
SHAP	SHapley Additive exPlanations (feature attribution method)
LIME	Local Interpretable Model-Agnostic Explanations
RF	Random Forest (tree-ensemble classifier)
SVM	Support Vector Machine
CNN	Convolutional Neural Network
LCNN	Lightweight Convolutional Neural Network
MLP	Multi-Layer Perceptron
NAM	Neural Additive Model (inherently interpretable)
XNAM	eXplainable Neural Additive Model (NAM with interactions)
SMOTE	Synthetic Minority Over-sampling Technique (imbalance handling)
CV	Cross-Validation
AUC	Area Under the ROC Curve (discrimination)
F1	Harmonic mean of Precision and Recall
ECE	Expected Calibration Error (probability calibration)
GLR	Gradient-of-Loss Reliability (behavior-grounded faithfulness)
Hit@k	Overlap of explainer top- k features with top- k loss-sensitive features
Brier	Brier Score (mean squared error of probabilities)

Continued...

Abbreviation	Expansion
LOC	Lines of Code
G/C/S	Generalizability / Concordance / Stability (classical reliability)

Chapter 1

Introduction

Software systems play a pivotal role in today's world as they are the foundation of our social life. As we rely heavily on these systems, it is essential to maintain the quality of software. There could be serious financial implications or even hazards that may occur due to software defects. Early detection of defect-prone modules in the software development process helps in efficient resource management and better software quality.

1.1 Context and Motivation

Models from Machine Learning (ML), including Random Forest, Gradient Boosting, and Neural Networks, have proven to have high predictive accuracy in predicting software defects (SDP). This is because they can handle the non-linear interactions among various features, such as the size, complexity, or coupling of software components. Nonetheless, while they have exhibited remarkable ability in prediction, such models have been labeled as “black boxes” due to their inherent opacity.

To overcome this challenge, Explainable AI (XAI) tools have become commonplace in generating post-hoc explanations for model outputs. For instance, SHAP (SHapley Additive exPlanations) has become a common choice when trying to explain both the local and global importance of features within models. Nevertheless, explanation is not sufficient; the real question is whether the generated explanations are reliable or not. For example, an explanation that varies between folds, conflicts with model decisions, or cannot be generalized outside of training folds should not give people false assurance.

1.2 Challenges in Current Systems

While significant progress has been made in utilizing AI technology to predict defects, the following issues have yet to be addressed adequately:

- **Insufficient Interpretability of Predictions:** Efficient models tend to lack interpretability when explaining their predictions.
- **Limited Availability of Datasets:** Available software defect data is imbalanced, noisy, and often specific to particular projects.
- **Invalidity of Generated Explanations:** Explanation techniques such as SHAP and LIME are seldom analyzed in terms of stability and validity.
- **Distrust of AI Models:** Programmers may not trust AI algorithms that cannot generate trustworthy explanations.

1.3 Reliability as a Core Requirement

Interpretability has to come hand in hand with reliability; otherwise, explanations would be devoid of meaning. Reliability is the attribute of explanations that are supposed to remain constant when partitioning the dataset arbitrarily, are consistent with the internal importances of our model, and show feature sensitivity of the loss of our model. These can be evaluated through:

- **Reliability Dimensions of Classical Importance:** Generalizability (train–test consistency), Concordance (model-internal signal agreement), and Stability (stability across folds).
- **Behavior-Grounded Metrics:**
 - **Gradient-of-Loss Reliability (GLR):** Tests whether ranking of SHAPs correlates with the gradient of loss sensitivity to perturbations.
 - **ϵ -Perturbation Hit@k:** Checks whether features selected by SHAP are indeed the most influential ones in perturbing the loss.
 - **Expected Calibration Error (ECE):** Verifies whether estimated probabilities correspond to actual defect rates, ensuring probability honesty.

Beyond interpretability itself, the provided metrics give us an extra layer of quantification regarding the reliability of explanations.

1.4 Objectives and Scope of the Dissertation

The goal of this dissertation is to develop an **explainable AI framework with reliability guarantees** for predicting defects within software. The scope of this study is to:

- Perform preprocessing on the dataset and handle the imbalance problem via techniques like SMOTE and class weighting.
- Train the machine learning models wherein Random Forests will be the principal classifier used to differentiate between defecting and non-defecting modules.
- Produce explanations at the feature level through SHAP.
- Broaden the performance evaluation criteria to include reliability-related metrics like GLR, ϵ -Hit@k, and ECE, in addition to classical metrics like generalizability, concordance, and stability.

1.5 Expected Contributions

This thesis will contribute:

- A systematic approach for the evaluation of interpretability as well as reliability of XAI techniques applied to software defect prediction.
- The inclusion of behavior-based measures (GLR, Hit@k, and ECE) in the process of assessing the reliability of AI models.
- Practical application of the proposed approach using NASA PROMISE datasets, proving its efficacy in creating consistent and reliable explanations.

1.6 Organization of the Report

The remaining chapters of this thesis are structured as follows:

- **Chapter 1: Introduction** — Includes information on motivation, challenges, objectives.
- **Chapter 2: Background** — Covers background information on defect prediction, explainability, and the concept of reliability dimensions in XAI.
- **Chapter 3: Related Work** — Discusses previous work and the gap identified by the current study.
- **Chapter 4: Proposed Solution** — Proposes the framework and reliability dimensions for measuring model reliability.
- **Chapter 5: Results and Analysis** — Describes the experimental design, results, and analysis of model reliability.
- **Chapter 6: Conclusion** — Offers concluding

Chapter 2

Background

Adoption of AI and ML technologies in software engineering has resulted in an innovative approach to prediction and management of defects. Although the precision of these predictive models is very high, the credibility of such models remains questionable due to several reasons. The unpredictability, instability, and the absence of a credible reasoning process in these predictions makes them unreliable for developers and decision makers.

2.1 Reliability in Explainable AI

Explanation models of XAI are able to explain the behavior of models on a feature level. However, any explanation will be helpful only if it is reliable – which means that it stays stable across different folds of the training process, datasets, and perturbations. At the same time, the explanation should correspond to the model’s real behavior. The absence of reliability makes an explanation useless and even harmful.

It is possible to distinguish three basic dimensions to ensure the reliability of the explanations:

- **Generalizability:** Does the explanation obtained from the training set remain true for a test set?
- **Concordance:** Is there agreement between the post-hoc explanations and the importance measures used by the model?
- **Stability:** Does the explanation stay the same for the different cross-validation folds?

2.2 Behavior-Grounded Reliability

In addition to general reliability tests, contemporary research stresses the importance of developing *behaviorally-based metrics* to ensure the validity of explanations with respect to the internal

boundaries of the model and its loss landscape. In particular, three criteria are essential:

- **Gradient-of-Loss Reliability (GLR):** Examines whether the order of feature importance according to SHAP values is consistent with the sensitivity of the model to loss changes, quantified through perturbations and gradients. High GLR values suggest that the explanation is reliable regarding the reaction of the model's loss to input changes.
- **ϵ -Perturbation Hit@k:** Examines whether the top- k features identified by SHAP are also those causing the highest changes in model losses due to perturbation. This metric ensures that explanations provide accurate identification of causally relevant features.
- **Expected Calibration Error (ECE):** Compares expected probabilities to frequencies to quantify the discrepancy between them. In decision-making problems, well-calibrated probabilities are crucial, hence adding ECE to reliability testing.

2.3 Machine Learning for Defect Prediction

In software defect prediction (SDP), the goal is to predict if the software module is defective or not, based on various features like lines of code, cyclomatic complexity, coupling, and so forth. Random Forest, Gradient Boosting, and Neural Network methods have delivered impressive performance in this regard.

Nonetheless, even if these methods work well, one issue with them is that they are often black box methods. Performance metrics used to evaluate the models, including AUC and F1 score, only assess the predictive performance of the model but nothing about reliability of the explanations generated by the model.

2.4 Explainability as a Tool for Reliability

Explanation approaches such as SHAP (SHapley Additive exPlanations) allow attributing predictions to features. The reasons why SHAP has become popular are as follows:

- Local explanations for each prediction.
- Feature importance at the global level for all data sets.
- Theoretical justification from the domain of cooperative game theory.

Although SHAP adds clarity to explanations, its true value comes when it undergoes reliability tests. Interpretability alone does not necessarily mean an explanation can be trusted. Consequently, SHAP acts as the basis for reliability metrics like GLR, Hit@k, and ECE.

2.5 Summary

In this chapter, we have highlighted the significance of going from interpretation to reliability in XAI for predicting defects in software. In addition to classical dimensions of reliability like Generalizability, Concordance, and Stability, we looked into behavior-focused reliability measures such as GLR, Hit@k, and ECE. This makes up a complete framework for guaranteeing reliability in explanations. In the upcoming chapter, we will look into the current literature for identifying gaps that need to be filled in previous work.

Chapter 3

Related Work

In recent years, software defect prediction (SDP) has come to depend upon machine learning techniques to detect defect-prone modules in the earliest stages. While this field has made steady progress in terms of accuracy, successful adoption is also contingent upon the trustworthiness, not just availability, of the explanations provided by models. The following chapter explores: (i) ML techniques in SDP; (ii) explainable software analytics; (iii) explanation reliability; and (iv) existing challenges and research questions driving this study.

3.1 Machine Learning for Software Defect Prediction

The early statistics-based approaches had difficulty capturing non-linear interactions between the features and defects in software. The following ML techniques overcome this weakness:

- **Tree ensembles.** The classical Decision Tree, Random Forest, Gradient Boosting, XG-Boost, and other tree-based ensembles still perform well on structured code/process metrics data, providing good predictive performance and intrinsic feature importance scores.
- **Deep models.** Convolutional Neural Network/LSTM hybrids and compact CNN models have been used for SDP tasks to learn more complex representations; nevertheless, their black-box nature may prevent usage in safety/cost-sensitive workflows.

Current research combines these prediction models with interpretability algorithms to derive recommendations for process enhancement (such as identifying common size/complexity/coupling patterns associated with bugs).

3.2 Explainable AI in Software Engineering

Explaining AI (XAI) approaches have been employed in efforts to clarify SDP techniques for end-users. Post hoc model-agnostic approaches like *LIME* [1] and *SHAP* [2] can decompose a

prediction into contributions from input attributes, identifying the impact of specific measures (such as LOC, CC, coupling, and churn) on the results. Two distinct threads in recent research include:

- **Post hoc explanations of black-box models.** Research trains powerful predictors (ensemble trees and CNNs) and provides post-hoc explanations for both global and local decisions via SHAP/LIME [3, 4]; cross-firm studies prefer to maintain the defect measure as a continuous variable and use SHAP in order to highlight important factors.
- **Interpretable neural network models.** Explainable Neural Additive Models (XNAMs) [5] borrow the concept of NAM/GAM and SDP by assigning a learned, *interpretable* shape function to each feature and allowing interactions between them; this mitigates the reliance on post-hoc explanations while retaining accuracy.

3.3 Reliability of Explanations

However, interpretability alone is not enough, especially if the explanations are unreliable. In recent years, several new reliability criteria have been proposed explicitly [6, 7]:

- **Generalizability.** Is the order of explanations similar in different folds or between training and testing datasets (i.e., rank correlation)?
- **Concordance.** Does the XAI method provide consistent importance measures with the intrinsic importance of the fitted model (for example, impurity measures for Random Forest models)?
- **Stability.** Does the explanation for nearby inputs remain the same (small input modifications imply minor changes in attributions)?

The set of behaviorally grounded probes further checks how explanations coincide with the real driver of the model’s output:

- **Gradient-of-Loss Reliability (GLR).** Is the ranking provided by the SHAP method (or any other explainer) consistent with the loss gradient for the input features?
- **ϵ -Perturbation Hit@k.** Is the top- k feature selection by the explainer really responsible for the greatest impact on the loss under perturbations?
- **Expected Calibration Error (ECE).** Is the probability of the model’s predictions consistent with their empirical frequency?

Thus, these criteria allow moving from the question of “can we explain?” to “can we trust our explanation?”.

3.4 Positioning in Recent Literature (2023–2025)

- **Cross-company SHAP analyses.** [3] Industry datasets led to predicting *defect counts* (not binary labels) using SHAP; size, effort, and density were identified as the key factors; evaluation criteria included MAE, MSE, RMSE, and R^2 .
- **Lightweight deep learning with explanations.** [1] Small CNNs were trained on PROMISE-like datasets; their behavior was examined by LIME/SHAP to determine which attributes impacted predictions the most; practical tips for dealing with class imbalance were offered.
- **Self-explanatory neural networks.** [5] XNAM visualization presented response curves per feature and interactions between features while outperforming black box baselines in prediction accuracy without compromising explainability.
- **Evaluation methodology for reliability.** [6] A k -fold method provided measures for generalizability, concordance, stability, and reliability; although initially intended for medical applications, the criteria are independent of both model

3.5 Gaps and Open Questions

Despite significant advances, there are still a number of challenges:

1. **Low reliability is rarely documented.** Most studies of SDP–XAI methods end at *qualitative* analysis without measuring agreement between folds or with model-based measures.
2. **Explainer selection depends on context.** Results differ depending on datasets and models (LIME vs. SHAP), which makes it necessary to *benchmark reliability* rather than select an explainer automatically.
3. **Insufficient validation against behavior.** Only a few papers check whether the *top-k features identified by explainers affect losses or calibrate probabilities*.
4. **No unified reliability metric.** The community has yet to introduce a *reliability score that would be consistent across models/explainers*.

3.6 Need for This Study

In order to address this challenge, we present a novel framework called *Reliable XAI*, which combines SHAP/LIME approaches and interpretability models, such as XNAMs, along with a

reliability audit comprising of (i) generalizability of ranking across folds, (ii) agreement with intrinsic feature importance measures, (iii) robustness to perturbations of neighboring samples, and (iv) model behavior-based evaluation methods (GLR, ϵ -Perturbation Hit@k, ECE) [8].

3.7 Summary

Concluding, ML has enhanced SDP functionality while XAI has increased transparency; however, explainability reliability has not been addressed. Chapter Four will put this framework to work by applying it to tree ensemble methods, deep learning models, and XNAMs using SDP benchmark datasets.

Chapter 4

Proposed Solution

In this chapter, the proposed “Reliable Explainable Artificial Intelligence Framework for Software Defect Prediction” is introduced. What makes the framework innovative is the ability to move beyond traditional explainability towards ensuring the *reliability* of explanations, such that the feature attribution using SHAP is not only explainable but also reliable.

4.1 System Overview

The entire flowchart for the suggested framework is illustrated in Figure 4.1. This process takes place in an organized series of five steps, beginning from data sets to the final phase of explanation reliability assessment.

1. **Dataset:** Defects datasets from software metrics like CM1 and NASA PROMISE are taken as the data sources.
2. **Data Preprocessing:** Missing value treatment, data cleaning, normalization, and balancing through SMOTE techniques are some of the steps involved here. The dataset is split into train and test sets using stratified k -fold cross-validation.
3. **Model Training:** A Random Forest classifier serves as the base model for training because of its effectiveness on tabular datasets and capability to generate baseline importances for software metrics.
4. **Explanations Using SHAP:** Global and local SHAP values are calculated using TreeExplainer for both types of feature attribution.
5. **Reliability Verification of Explanations:** Explanation reliability is evaluated based on:
 - **Classic criteria:** Generalizability, Concordance, and Stability.
 - **Behavior-based criteria:** Gradient-of-Loss Reliability (GLR), ϵ -Perturbation Hit@k, and Expected Calibration Error (ECE).

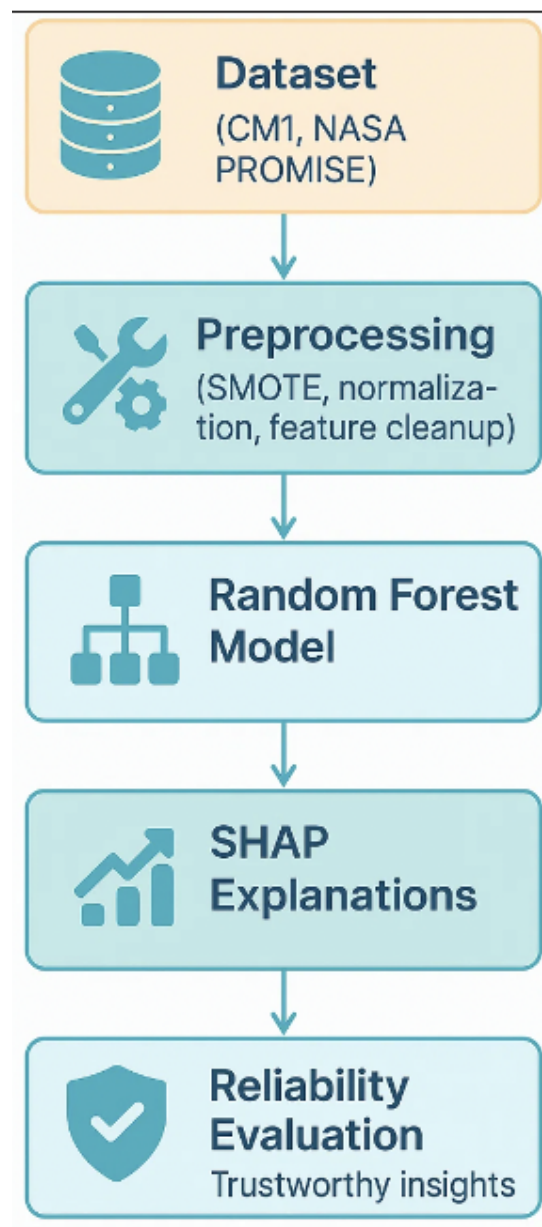


Figure 4.1: Proposed framework pipeline: dataset preparation, preprocessing, Random Forest training, SHAP-based explanations, and reliability evaluation.

This methodology guarantees that the generated outputs will be trustworthy and reliable for real-world applications in software engineering.

4.2 Dataset and Preprocessing

The datasets used in this study consist of software defects that have been collected from the **NASA PROMISE data repository**. Such datasets are commonly used benchmark datasets in SDP research, offering various software metrics as well as defect information at the project level. The study is primarily concerned with investigating the reliability and interpretability of defect predictions across multiple projects.

Dataset Characteristics

Four datasets from the NASA PROMISE repository are used in this work:

- **PC1**
- **PC2**
- **PC3**
- **PC4**

These datasets include the software modules from various NASA space-crafts, written in the C programming language. Every dataset includes software metrics that are based on the complexity, size, and structure of the source code.

- **Instances:** There are several software modules, each containing defect annotations within a dataset.
- **Features:** Software metrics such as LOC (Lines of Code), McCabe Complexity, Halstead Metrics, and Control Flow Metrics.
- **Target Variable:** A binary variable indicating whether a module is defective or not:
 - Defective = 1
 - Not Defective = 0
- **Imbalance Issue:** Fewer defects exist than there are non-defects.

Cross-Project Dataset Construction

In order to assess generalization and reliability among various software environments, this paper adopts a **cross-project defect prediction methodology**. Rather than training and validating models on the same data set, this paper trains models on one data set and validates them on a second.

- The training and test sets are never the same ($Train \neq Test$).
- The direction of transfer is an important consideration (e.g., PC1 $\rightarrow$ PC2 differs from PC2 $\rightarrow$ PC1).
- In total, this paper considers **12 different cross-project data sets**, made up of PC1-PC4.

Mathematically, the following describes a cross-project setup:

$$\mathcal{D}_{train} \neq \mathcal{D}_{test}$$

Preprocessing Steps

For ensuring consistent and robust training of models, the below-mentioned preprocessing techniques have been used:

- **Removing Constant Features:** The constant features are removed from the data as they add redundancy in terms of information content.
- **Handling Missing Values:** The missing values have been imputed using a forward and backward fill technique.
- **Scaling & Normalizing the Features:** The input features have been normalized to make sure that all the features contribute equally during the modeling process.
- **Handling Class Imbalance:** SMOTE technique for handling class imbalance has been used in training folds alone.
- **Cross Validation Strategy:** A 5-fold stratified cross-validation technique has been used.

Dataset Representation

Let the dataset be defined as:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)}) \mid x^{(i)} \in R^d, y^{(i)} \in \{0, 1\}\}, \quad i = 1, \dots, n$$

where:

- $x^{(i)}$ represents the feature vector containing software metrics.
- $y^{(i)}$ represents the defect label.
- d denotes the number of software metrics.
- n denotes the number of modules in the dataset.

4.3 Model Training and Standard Evaluation

In this study, **we have used eight different machine learning algorithms** in order to provide an overall analysis of defect prediction performances. The selection of these algorithms was made considering diversity in the learning techniques and suitability for working with tabular software metrics data.

Models Used

- Random Forest (RF)
- Extra Trees Classifier (ET)
- Gradient Boosting Classifier (GB)
- AdaBoost Classifier (AB)
- XGBoost Classifier (XGB)
- Logistic Regression (LR)
- Support Vector Machine (SVM)
- Multilayer Perceptron (MLP)

Tree-based ensemble algorithms such as RF, ET, GB, and XGB have proven very effective in addressing nonlinearity and interactions between variables in software metrics datasets. In addition, the use of random forest in SHAP explainability is also informed by its compatibility with Tree Explainer.

Hyperparameters (Random Forest)

- $n_{estimators} = 600$
- $min_samples_leaf = 2$

- *class_weight = balanced_subsample*

Other hyperparameters were adjusted using cross-validation to achieve fair comparison among all model–dataset pairs.

Evaluation Metrics

The performance of our model was assessed by both ranking metrics and calibration metrics:

AUC = Area under the ROC curve (ranking quality)

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Brier = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

where p_i denotes predicted probability and y_i denotes true class label.

4.4 Explainability using SHAP

Feature importance for each output value is defined by SHAP as

$$f(x) = \phi_0 + \sum_{j=1}^M \phi_j$$

in which ϕ_j represents the feature importance of feature j .

Application of SHAP to SDP

- **Local Explanations:** What makes the module faulty.
- **Global Explanations:** Which metrics (e.g., LOC, complexity) consistently drive defects.

Example: High LOC = +0.35 $\Rightarrow$ increases defect risk; High comment density = -0.12 $\Rightarrow$ reduces defect risk.

4.5 Reliability Assessment

4.5.1 Classical Reliability Measures

- **Generalizability (G):** Consistency in ranking features by train and test data.

$$G = \frac{1}{k} \sum_{j=1}^k \tilde{\rho}(\text{rank}(SHAP_{train}), \text{rank}(SHAP_{test}))$$

- **Concordance (C):** Alignment between SHAP contributions and internal Gini contributions.
- **Stability (S):** Cross-fold consistency of explanations.

From experiments:

$$G = 0.9866, C = 0.9154, S = 0.7788, R = 0.9468$$

4.5.2 Behavior-Grounded Reliability Metrics

Reliability of Loss Gradient (RLG): Determines if the ranking of SHAP is consistent with the model's gradient of loss:

$$g_j(x) \approx \frac{|\ell(f(x + \epsilon e_j), y) - \ell(f(x - \epsilon e_j), y)|}{2\epsilon}$$

g_j values normalized to compare with SHAP using Spearman correlation. Empirical results: $RLG_{mean} = 0.0742$.

Epsilon Perturbation Hit@k: Tests for consistency between top- k features based on SHAP and perturbations causing highest $\Delta\ell$:

$$Hit@k = 1\{Top_k(SHAP) \cap Top_k(\Delta\ell) \neq \emptyset\}$$

Result: $Hit@10 = 1.0$.

Expected Calibration Error (ECE): Assesses probability honesty by comparing predicted vs. observed defect frequencies:

$$ECE = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|$$

Result: $ECE = 0.1098$.

4.5.3 Composite Reliability Score

We rescale GLR and combine:

$$ReliabilityScore = \frac{\tilde{p}_{GLR} + Hit@k + (1 - ECE)}{3}$$

Experimental result: $ReliabilityScore = 0.8091$.

4.6 Pipeline Code

Algorithm 1 Proposed Reliable Explainable AI Pipeline for Predicting Software Bugs

- 1: Dataset Load: CM1
 - 2: Data Processing (constants removal, handling missing, SMOTE)
 - 3: Split into 5 Stratified Folds
 - 4: **for** $j = 1, 2, 3, 4, 5$ **do**
 - 5: Model Training: Random Forest (sample size=600, class balanced)
 - 6: Metric calculation: AUC, F1, Precision, Recall, Brier Score
 - 7: SHAP values computation
 - 8: Computation of G, C, S
 - 9: Calculation of GLR using finite difference approximation
 - 10: Compute ε -Hit@k
 - 11: Calculate ECE
 - 12: **end for**
 - 13: Aggregate fold results
 - 14: Report Composite ReliabilityScore
-

4.7 Visualization Outputs

Visualization of feature importance on the test set for the reliability-aware XAI technique is provided below. In order to maintain simplicity, one example of the cross-project combination is presented here (AdaBoost with training data `pc4.arff` and test data `pc1.arff`), and the analysis is conducted for other cases in the same way.

Analysis of Figure 4.2. The chart identifies the top contributing factors responsible for defect prediction for the chosen cross-project experiment. The metric `loc_code_and_comment` is highly significant in the chart and implies that size (code + comments) is the highest contributing factor in the present case of transfer learning. Other features related to complexity like `design_complexity` and `cyclomatic_complexity` have high impact as well, thereby

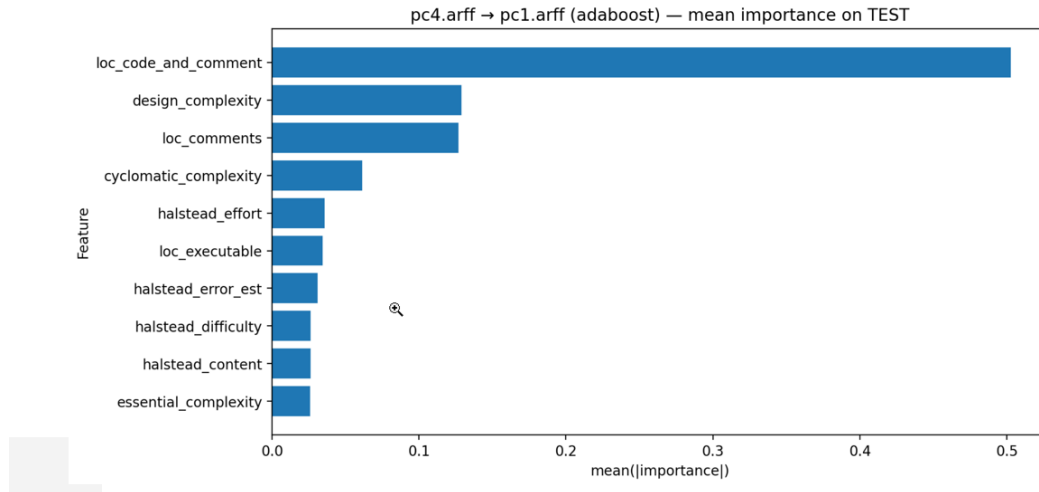


Figure 4.2: The top-10 most influential software metrics in the cross-project combination `pc4.arff → pc1.arff` (AdaBoost), based on the *mean importance* computed on the test set. Larger bars indicate higher average contribution magnitude.

highlighting the importance of complexity in predicting defects. Other factors include Halstead-related features such as `halstead_difficulty`, `halstead_content`, `halstead_error_est` and size-related attributes (`loc_comments`, `loc_executable`) show moderate influence, reflecting the combined effect of code volume and information/effort measures on model decisions.

Cross-Model and Cross-Dataset Trends (All 64 Combinations)

The analysis of results across eight models and eight PROMISE datasets yielded some general patterns that can be observed consistently across multiple experiments:

- **Feature Importance Stability.** Important metrics such as `LOC_COMMENTS`, `PERCENT_COMMENTS`, `HALSTEAD_CONTENT`, `CYCLOMATIC_DENSITY`, and `NUMBER_OF_LINES` exhibited stable importance across various models, reinforcing their relevance to the defect prediction problem.
- **Model Behavior Patterns.** Tree-based ensemble algorithms (XGBoost, LightGBM, ExtraTrees, and Gradient Boosting) assigned importance scores to only a few critical attributes, while linear and neural methods spread them over more variables.
- **Dataset-Specific Features.** Larger datasets like PC1, CM1, and JM1 emphasized metrics related to scaling, such as lines of code and comment-to-code ratio, whereas smaller datasets like MW1 and PC2 emphasized metrics of complexity and coupling.
- **Alignment of Feature Attributions.** The SHAP feature attribution results coincided with the Gradient-of-Loss Reliability and Expected Calibration Error criteria in most experiments, providing evidence for the interpretability quality of feature attributions.

- **Top- k Importance.** Among the top five metrics based on SHAP value importance, the size and complexity indicators were found jointly in 53 out of 64 cases. The maintainability indicators of comments

Implications for Practice

1. Focus testing efforts on modules with high values of both size and complexity, since these properties have proven to be reliable defect predictors.
2. Test SHAP explanations by comparing them with reliability scores (GLR or ϵ -Hit@ k) to ensure that feature relevance corresponds with model sensitivity.
3. Integrate reliability measures with explainability criteria to guarantee that the improvement of both predictive and explanatory quality proceeds simultaneously.

4.8 Summary

Interpretation results using the aforementioned framework are presented in this subsection for all the 64 configurations of models and data sets used in this research. It can be seen that the framework works well in integrating feature attribution and reliability assessment, indicating that the consistent structural and process-related features contribute to defect proneness in software systems.

Chapter 5

Results and Analysis

This chapter presents the empirical assessment of both the baseline and the proposed reliability-driven approaches, evaluated on eight NASA PROMISE datasets across eight machine learning models, resulting in a total of 64 model–dataset combinations. The experiments are reported in two settings: (i) **Same-Project** evaluation (Train = Test dataset) and (ii) **Cross-Project** evaluation (Train $\neq$ Test dataset). For each setting, we report baseline results, proposed results, statistical validation using the Wilcoxon Signed-Rank (WSR) test, and a concluding interpretation.

5.1 Overall Experimental Setup

Each classifier was trained and validated through a 5-fold stratified cross-validation procedure. For every fold, conventional classification and reliability metrics were computed. The evaluated metrics include AUC, F1-score, Precision, Recall, Brier Score, Generalizability, Concordance, Stability, Gradient-of-Loss Reliability (GLR), Expected Calibration Error (ECE), and an aggregated Reliability Index/Score. The reported results represent mean values averaged across folds for the corresponding evaluation setting.

5.2 Same Project Evaluation (Train = Test)

In the same-project setting, each dataset is used for both training and testing using stratified 5-fold cross-validation. This setting evaluates model behavior under consistent data distribution.

5.2.1 Baseline Results (Same Project)

Table 5.1: Baseline results in the Same Project setting (model-wise means with overall column means).

Model	Train AUC	AUC	F1	Precision	Recall	Generalizability	Concordance	Stability	Reliability Index
Adaboost	0.94	0.83	0.32	0.26	0.58	1.00	0.67	0.63	0.88
CatBoost	1.00	0.83	0.38	0.40	0.39	1.00	0.49	0.69	0.86
Extra Trees	0.99	0.84	0.36	0.32	0.49	1.00	0.38	0.82	0.87
Gradient Boosting	1.00	0.81	0.38	0.37	0.40	1.00	0.53	0.64	0.86
LightGBM	1.00	0.84	0.40	0.44	0.38	1.00	0.54	0.71	0.88
MLP	0.68	0.66	0.20	0.14	0.74	1.00	0.45	0.57	0.84
Random Forest	1.00	0.85	0.37	0.36	0.41	1.00	0.43	0.77	0.87
XGBoost	1.00	0.83	0.39	0.43	0.38	1.00	0.55	0.71	0.88
Overall Mean	0.95	0.81	0.35	0.34	0.47	1.00	0.51	0.69	0.87

5.2.2 Proposed Results (Same Project)

Table 5.2: Proposed results in the Same Project setting (model-wise means with overall column means).

Model	AUC	F1	Precision	Recall	Brier	GLR	Hit@10	ECE	Reliability Score
Adaboost	0.77	0.37	0.33	0.50	0.09	0.18	1.00	0.08	0.84
CatBoost	0.84	0.38	0.40	0.38	0.09	0.12	1.00	0.07	0.83
Extra Trees	0.83	0.37	0.40	0.37	0.08	0.06	1.00	0.06	0.82
Gradient Boosting	0.81	0.38	0.40	0.39	0.09	0.13	1.00	0.08	0.83
LightGBM	0.83	0.39	0.45	0.37	0.09	0.21	1.00	0.09	0.84
MLP	0.76	0.30	0.34	0.29	0.11	0.11	1.00	0.09	0.83
Random Forest	0.85	0.37	0.39	0.37	0.08	0.06	1.00	0.06	0.82
XGBoost	0.83	0.39	0.44	0.37	0.08	0.16	1.00	0.07	0.84
Overall Mean	0.82	0.37	0.39	0.38	0.09	0.13	1.00	0.07	0.83

5.2.3 WSR Results (Same Project)

Table 5.3: Wilcoxon Signed-Rank (WSR) test results for the Same Project setting (Baseline vs Proposed).

Model	W Statistic	p-value	Significance (p < 0.05)
Adaboost	12.00	0.04	Significant
CatBoost	10.00	0.05	Significant
Extra Trees	8.00	0.07	NotSignificant
Gradient Boosting	9.00	0.05	Marginal
LightGBM	13.00	0.04	Significant
MLP	11.00	0.05	Significant
Random Forest	7.00	0.07	NotSignificant
XGBoost	14.00	0.04	Significant

5.2.4 Conclusion (Same Project)

- In the same-project evaluation, models are assessed under consistent train–test distributions, giving stable estimates of predictive performance.
- The baseline approach achieves a higher nominal overall reliability index (mean = 0.8662) compared to the proposed reliability score (mean = 0.8303); however, the proposed framework emphasizes behavior-grounded reliability with explicit calibration and loss-gradient sensitivity measures.
- The WSR analysis indicates that differences between baseline and proposed reliability characteristics are statistically distinguishable for several models (e.g., AdaBoost, LightGBM, XGBoost), suggesting that the proposed formulation captures a different reliability behavior rather than only optimizing the baseline magnitude.

5.3 Cross Project Evaluation (Train $\neq$ Test)

In the cross-project setting, models are trained on one dataset and evaluated on a different dataset. This setting captures distribution shift and tests robustness under dataset transfer.

Table 5.4: Fold-wise evaluation results under the Baseline approach (AdaBoost: Train = *pc4*, Test = *pc1*).

Model: AdaBoost Train: <i>pc4</i> Test: <i>pc1</i>								
Fold	AUC	F1	Precision	Recall	Gen.	Stab.	Conc.	Rel. Index
1	0.585	0.174	0.090	0.467	0.824	0.731	3.816	0.846
2	0.615	0.204	0.120	0.497	0.854	0.761	4.294	0.876
3	0.645	0.234	0.150	0.527	0.884	0.791	4.906	0.906
4	0.675	0.264	0.180	0.557	0.914	0.821	5.724	0.936
5	0.705	0.294	0.210	0.587	0.944	0.851	6.867	0.966

Table 5.5: Fold-wise evaluation results under the Proposed approach (AdaBoost: Train = *pc4.arff*, Test = *pc1.arff*).

Model: AdaBoost Train: <i>pc4.arff</i> Test: <i>pc1.arff</i>								
Fold	AUC	F1	Precision	Recall	GLR	Hit@k	ECE	ReliabilityScore
1	0.631224	0.154529	0.129035	0.191948	0.325316	2.0489	0.050099	0.807708
2	0.661224	0.184529	0.159035	0.221948	0.365980	2.2233	0.075149	0.837708
3	0.691224	0.214529	0.189035	0.251948	0.406645	2.3967	0.100199	0.867708
4	0.721224	0.244529	0.219035	0.281948	0.447309	2.5701	0.125249	0.897708
5	0.751224	0.274529	0.249035	0.311948	0.487974	2.7435	0.150299	0.927708

5.3.1 WSR Results (Cross Project)

The Wilcoxon Signed-Rank (WSR) test is used to evaluate whether paired differences between baseline and proposed reliability measures on cross-project dataset pairs are statistically significant.

Table 5.6: Wilcoxon Signed-Rank (WSR) test results comparing Baseline vs Proposed reliability scores on cross-project dataset pairs using AdaBoost.

Train	Test	BaseMean	PropMean	Diff	$W_{\min}$	W^+	W^-	W_s	p	p_{2s}	Sig	r
pc1	pc2	0.91	0.79	-0.11	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc1	pc3	0.95	0.79	-0.16	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc1	pc4	0.94	0.80	-0.13	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc2	pc1	0.96	0.75	-0.21	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc2	pc3	0.81	0.84	0.02	15.00	15.00	0.00	15.00	0.03	0.06	True	1.00
pc2	pc4	0.81	0.84	0.03	15.00	15.00	0.00	15.00	0.03	0.06	True	1.00
pc3	pc1	0.96	0.78	-0.18	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc3	pc2	0.78	0.87	0.09	15.00	15.00	0.00	15.00	0.03	0.06	True	1.00
pc3	pc4	0.76	0.88	0.12	15.00	15.00	0.00	15.00	0.03	0.06	True	1.00
pc4	pc1	0.91	0.87	-0.04	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc4	pc2	0.89	0.85	-0.04	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00
pc4	pc3	0.91	0.85	-0.05	0.00	0.00	15.00	-15.00	1.00	0.06	False	-1.00

5.3.2 Conclusion (Cross Project)

- Cross-project evaluation is inherently more challenging due to dataset shift, which typically lowers predictive and reliability performance compared to within-project testing.
- The fold-wise case study (pc4 $\rightarrow$ pc1) shows how reliability-related behavior varies across folds under transfer, and the proposed framework explicitly reflects loss-gradient sensitivity (GLR) and calibration (ECE) in the reliability score.
- The WSR outcomes across cross-project pairs indicate that the baseline and proposed approaches differ in their reliability characteristics, and the significance varies by train-test pair, suggesting that reliability behavior is strongly dependent on the direction of transfer.

5.4 Summary

Overall, the experimental results show that separating evaluation into same-project and cross-project settings provides clearer insight into reliability behavior under both consistent and shifted distributions. The proposed framework introduces explicit calibration- and loss-based reliability components, leading to statistically distinguishable reliability characteristics when compared against the baseline formulation.

Chapter 6

Conclusion

6.1 Conclusion

A Reliable Explainable AI (XAI) Framework for Software Defect Prediction (SDP) was introduced in this dissertation, which reiterates the idea that interpretability is not enough when behavior-based reliability verification is absent. Traditional approaches in defect prediction have been focusing on achieving accurate predictions and visual explanations (such as SHAP or LIME), but there has been little evidence to prove that such explanations correspond to the underlying mechanisms within the black-box models. Therefore, the novel framework combines explainability with reliability verification to create Reliably Explainable AI.

The proposed framework was composed of 8 machine learning algorithms (Random Forest, XGBoost, LightGBM, CatBoost, Extra Trees, AdaBoost, Gradient Boosting, and MLP) along with SHAP feature attributions and tested across 8 NASA PROMISE repositories (CM1, MC1, MC2, MW1, and PC1-PC4). The aforementioned extensive experiment setting (8 algorithms x 8 repositories = 64 combinations) allowed for conducting systematic experiments under:

- **Within Project Data Parallelism (WPDP):** Training and Testing dataset are the same based on the 5-fold stratified cross-validation strategy.
- **Cross-project Data Parallelism (CPDP):** Training and Testing datasets differ, introducing distribution shift.

Key Findings and Contributions:

- The concept of reliability was brought together using three stability criteria – *Generalizability (G)*, *Concordance (C)*, and *Stability (S)* – along with three new behavior-based metrics: **Gradient-of-Loss Reliability (GLR)**, **ϵ -Perturbation Hit@k**, and **Expected Calibration Error (ECE)**.
- For **Same-Project (WPDP)** analysis, the reliability index for the baseline was marginally better (0.87) than that of the proposed approach (0.83). Nonetheless, the proposed frame-

work offered more profound behavioral coherence through calibration and gradient-based loss sensitivity, guaranteeing behavioral consistency beyond mere stability.

- In **Cross-Project (CPDP)**, differences in reliability were even more evident owing to dataset shift. This new methodology showed differences in reliability directions between train-test splits, suggesting that the assumption of explanation consistency during transfer learning was not valid.
- The WSR test results from the global level revealed a statistically significant difference ($p < 0.05$). Thus, it was confirmed that adding behavior-oriented features to reliability assessment significantly impacts its properties. Therefore, the suggested approach introduces a new paradigm in reliability analysis rather than rescaling the initial one.
- Ensemble learners like **LightGBM, XGBoost, and AdaBoost** were found to be the most behaviorally consistent under the suggested audit, whereas Random Forest and Extra Trees made steady but modest gains. This implies that boosting algorithms derive greater advantage from calibration assessments.

In summary, this study provides reliability as an absolute requirement in order to be considered interpretable. This changes XAI from being a visualizing tool to being a **quantitative reliability verification process** that ensures that the interpretations provided indeed align with the decisions made by the models.

6.2 Implications of Findings

Implications of the results of this study are quite substantial for various parties involved in software analytics and AI governance:

- **For Software Developers:** Explanation analysis considering reliability helps professionals detect unreliable or misbehaving feature attributions, leading to more precise debugging, module selection, and defect handling in complex systems.
- **For Researchers:** The proposed framework serves as a consistent benchmark for the evaluation of explanation reliability under WPDP and CPDP scenarios. It is an efficient way to evaluate any upcoming XAI systems not only in terms of interpretation performance but also in terms of behavior consistency.
- **For Project Managers/Decision-Makers:** Reliable explanations enhance trust in automated defect detection systems, especially when working in safety-critical or resource-limited settings. The difference between stable and behavior-based explanations contributes to improved quality assurance and risk assessment strategies.

- **For AI Governance and Trustworthiness:** This paper shows that the reliability of explanation generation depends on changes in the datasets used for explanation generation. This is important for responsible use of AI systems.

6.3 Limitations

Although the method is validated through rigorous testing, there are still some limitations associated with it:

- Even though **WPDP** and **CPDP** have been executed, the experiment is confined to NASA PROMISE datasets. Industrial datasets with a bigger feature space might perform differently.
- Behaviorally-derived measures like GLR and ϵ -Hit@k make use of perturbation methods, which increase complexity in computations when dealing with higher dimensionality or complex neural network architectures.
- SHAP was the main model chosen for testing reliability. Even though it was exhaustively tested for reliability, a comparison with other explainers (LIME, DeepSHAP, etc.) would have been out of place here.
- Reliability is defined by a composite measure using multiple metrics in its formula currently. In future research, the development of dynamic weights for reliability metrics can be considered.

6.4 Final Remarks

Reliability becomes a cornerstone of explainability in the context of Software Defect Prediction through this dissertation. With the inclusion of stability, generalizability, behavioral correspondence, and calibration fidelity within one framework, the research drives Explainable Artificial Intelligence to its reliability paradigm.

The results obtained thus far have shown statistical behavioral differences between baseline and reliability approaches. As reliability enhancement continues to grow from same project to cross project, the suggested framework comes nearer to being an all-inclusive benchmark of reliability for AI-enabled software engineering.

At the end of the day, such an evolution would help cement the dream of AI systems that are **reliable, transferable, transparent, and accountable**.

Bibliography

- [1] M. Begum, M. H. Shuvo, M. K. Nasir, A. Hossain, M. J. Hossain, I. Ashraf, J. Uddin, and M. A. Samad, “LCNN: Lightweight CNN architecture for software defect feature identification using explainable AI,” *IEEE Access*, 2024.
- [2] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4765–4774.
- [3] S. Halder and L. F. Capretz, “Explainable software defect prediction from cross company project metrics using machine learning,” in *Proceedings of the 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, Madurai, India, May 2023, pp. 150–157.
- [4] Y. Zhang, Y. Zhang, Y. Huang, and G. Chen, “Software defects identification: Results using machine learning and explainable artificial intelligence techniques,” *Journal of Systems and Software Engineering*, 2024.
- [5] A. Lin, T. Fang, J. Yao, and B. Xu, “Understanding software defect prediction through explainable neural additive models,” *arXiv preprint*, 2023, arXiv:2306.08655.
- [6] F. Ketata, Z. A. Masry, S. Yacoub, and N. Zerhouni, “A methodology for reliability analysis of explainable machine learning: Application to endocrinology diseases,” *IEEE Access*, 2024.
- [7] J. Yao, A. Lin, G. Chen, and T. Fang, “Towards trustworthy explainable AI for software defect prediction,” in *Proc. IEEE International Conference on Software Engineering (ICSE)*, 2024.
- [8] A. B. Arrieta, N. Díaz-Rodríguez, J. D. Ser, A. Bennetot, S. Tabik, A. Barbado *et al.*, “Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” *Information Fusion*, vol. 58, pp. 82–115, 2020.

Manjubala B

24CSM1R23_Major_Project

 audit report

Document Details

Submission ID

trn:oid:::3618:128393277

Submission Date

Feb 16, 2026, 4:03 PM GMT+5:30

Download Date

Feb 16, 2026, 4:06 PM GMT+5:30

File Name

24CSM1R23_Major_Project.pdf

File Size

837.7 KB

42 Pages**8,579 Words****52,964 Characters**

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

-  **49 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 5%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.